

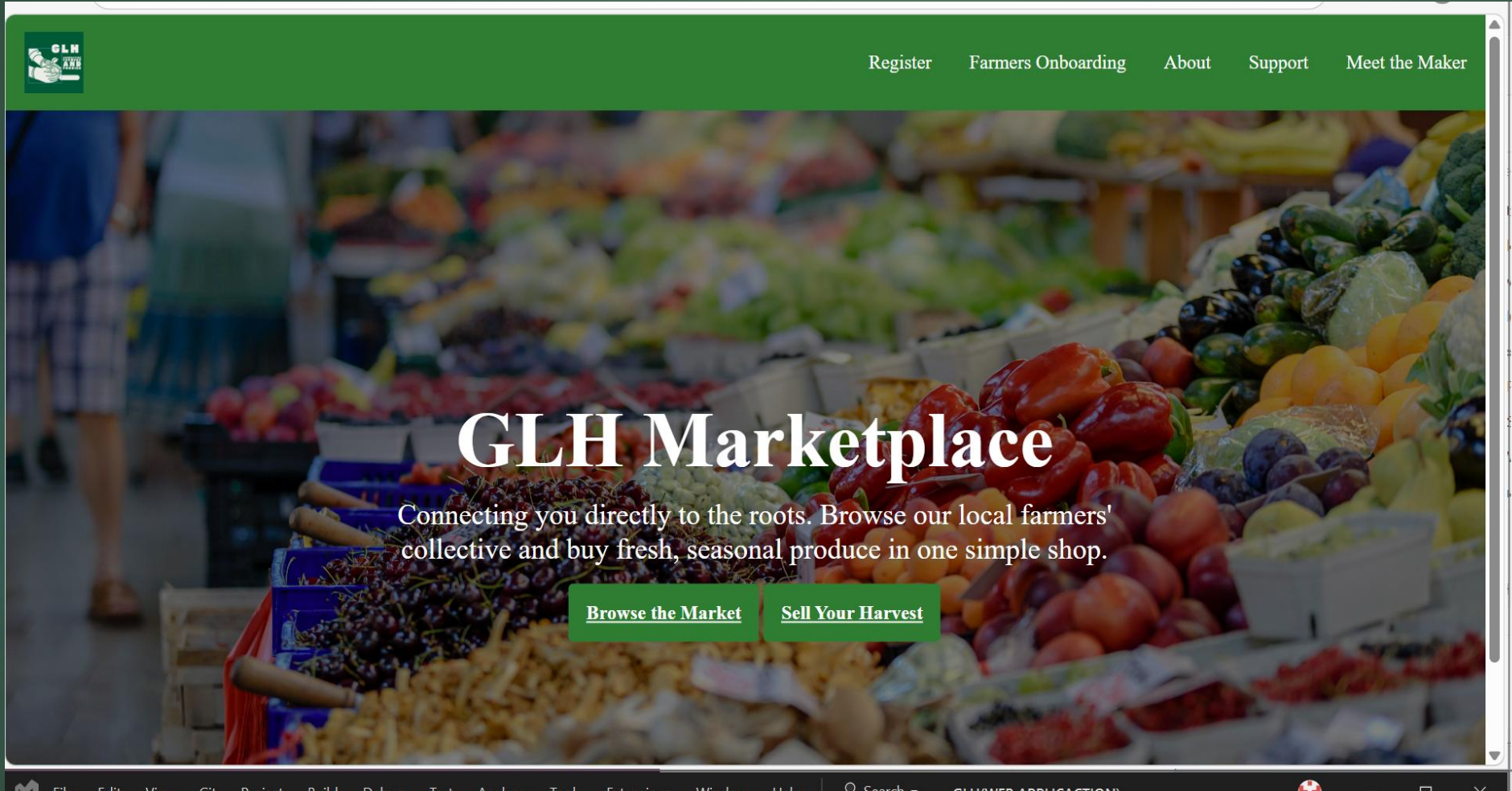


# Prototype's functionalities

## Technical audience

BY ASHLEY KARIRI

# The home page



- 
- ▶ On the home page there are seven linked pages that a user can go to without the need of signing in which are:
    - ▶ Register, Farmer Onboarding , About , Support , Meet the maker , Browse the market, Sell your harvest
  - ▶ These pages do not hold any of the users information and do not require any sort of information as they have nothing



# Register page

## Register

**Username**

**Password**

Sign Up

[Do you have an account already.](#)

- ▶ The register page is where the user enters their preferred user name and a password
- ▶ The system cross checks if the username is taken by other account and if it is it will give the error "This username is reserved for staff."

```
@app.route('/register', methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")
        db = get_db()

        # 1. Check if name is taken by an ADMIN first (Safety Check)
        existing_admin = db.execute("SELECT 1 FROM admins WHERE Admin_username = ?", (username,)).fetchone()
        if existing_admin:
            return render_template("register.html", error="This username is reserved for staff.")

        try:
            # 2. Hash the password and save the new customer
            hashed_pw = generate_password_hash(password, method="pbkdf2:sha256")
            db.execute("INSERT INTO users (username, password) VALUES (?, ?)", (username, hashed_pw))
            db.commit()

            # 3. Clear any old session and set as 'customer'
            session.clear()
            session['username'] = username
            session['is_farmer'] = False # They are a customer, not a farmer
            session['role'] = 'customer'

            flash("Account created! Welcome to the shop.")
            print('you are in ')
            return redirect(url_for("home")) # Send customers to the shop home page

        except sqlite3.IntegrityError:
            return render_template("register.html", error="Username already taken!")

    return render_template("register.html")

def login_required_farmers(f):
```

- If the username is free then the password is hashed/encrypted and the username and password get sent to the database and if accessed the password will be hashed and can not be used to enter if the password was the wrong one

```
try:
    # 2. Hash the password and save the new customer
    hashed_pw = generate_password_hash(password, method="pbkdf2:sha256")
    db.execute("INSERT INTO users (username, password) VALUES (?, ?)", (username, hashed_pw))
    db.commit()

# 3. Clean any old session and set as localhost
```

|                     |                                                   |
|---------------------|---------------------------------------------------|
| > order_items       | CREATE TABLE order_items ( id INTEGER PRIMARY KEY |
| > orders            | CREATE TABLE orders ( id INTEGER PRIMARY KEY AUT  |
| > product_movements | CREATE TABLE product_movements ( movement_id IN   |
| > sqlite_sequence   | CREATE TABLE sqlite_sequence(name,seq)            |
| > users             | CREATE TABLE users ( id INTEGER PRIMARY KEY AUT   |
| Indices (0)         |                                                   |
| Views (0)           |                                                   |
| Triggers (0)        |                                                   |



# Farmers Onboarding

## Join our other farmers

Username:

First Name:

Second Name:

Farm Name:

Phone Number:

Email:

Date of Birth:

Set Password:

Go Back

Sign Up

- ▶ This page will require a lot more information than that of the normal log in page it will require the user to enter farm name, phone number birth day which is some of the information needed for legal purposes and they all get sent to the database to be stored
- ▶ The password is hashed/encrypted
- ▶ And if the user name is taken correct procedures will take place

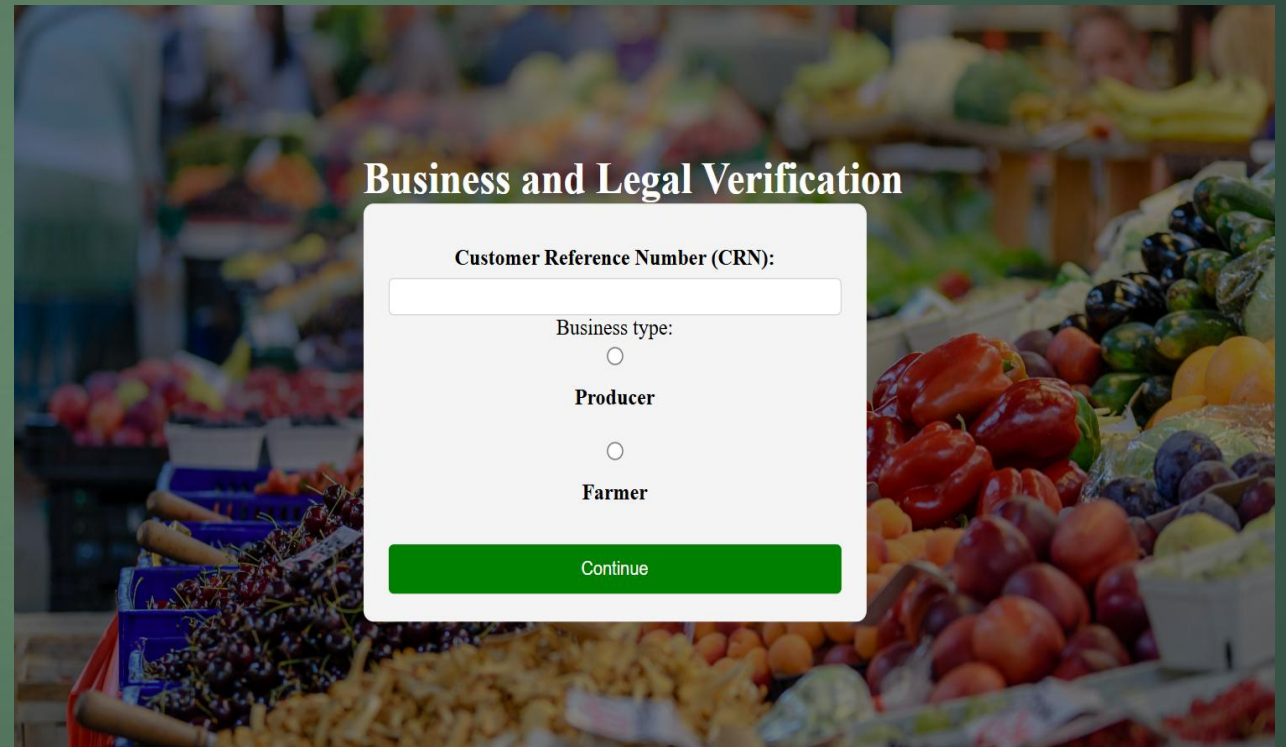
```
        return redirect(url_for('legalusersvarification'))
    except sqlite3.IntegrityError:
        flash("Error: That username or email is already taken.")
        return render_template("FarmersOnBoarding.html")
    except Exception as e:
        print(f"Error: {e}")
        return f"An unexpected error occurred: {e}"

return render_template('FarmersOnBoarding.html')
```

```
0
1 @app.route('/FarmersOnBoarding', methods=['GET', 'POST'])
2 def FarmersOnBoarding():
3     if request.method == 'POST':
4         # DEFINE DB FIRST
5         db = get_db()
6
7         username = request.form.get('username')
8         password = request.form.get('password')
9         fname = request.form.get('firstname')
10        sname = request.form.get('secondname')
11        farm = request.form.get('farm_name')
12        phone = request.form.get('phone_number')
13        email = request.form.get('email')
14        dob = request.form.get('dob')
15
```



- After they pass through the first information requirement page they are taken to the second one where it will ask whether, they are a farmer or a producer and their customer reference number that every farmer/producer who sells goods needs to have



**Business and Legal Verification**

Customer Reference Number (CRN):

Business type:

☐ Producer

☐ Farmer

[Continue](#)

# GLH Farmer Portal

The screenshot displays the GLH Farmer Portal interface. At the top, a green navigation bar contains the text "GLH Farmer Portal" on the left and "Home", "My Profile ▼", and "Logout" on the right. Below the navigation bar, a white content area features a large heading "Welcome back, farmtest1!" followed by the subtext "Manage your farm operations and live inventory below." Below this, five rounded rectangular buttons are arranged in two rows. The first row contains "Stock Management" (Update or edit current listings.), "Inventory" (View total stock levels across all categories.), and "Move Product" (Transfer items to storage or GLH facilities.). The second row contains "Order Fulfilment" (Pack today's customer orders.) and "Contact GLH" (Direct support for GLH farmers.). On the right side of the interface, a vertical sidebar is partially visible, showing a list of numbers from 6 to 10, with the number 10 highlighted in red.

GLH Farmer Portal

Home My Profile ▼ Logout

## Welcome back, farmtest1!

Manage your farm operations and live inventory below.

**Stock Management**  
Update or edit current listings.

**Inventory**  
View total stock levels across all categories.

**Move Product**  
Transfer items to storage or GLH facilities.

**Order Fulfilment**  
Pack today's customer orders.

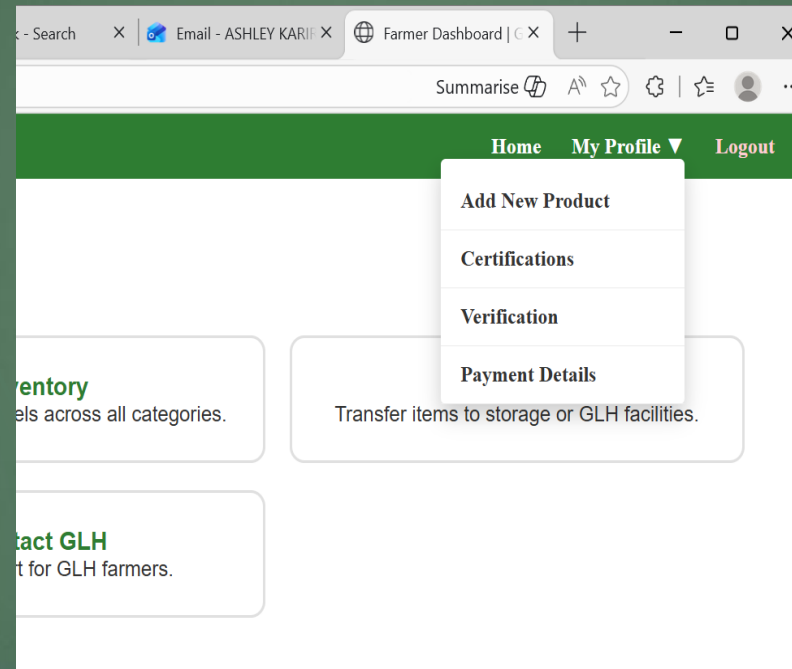
**Contact GLH**  
Direct support for GLH farmers.

6  
7  
8  
9  
10

- 
- ▶ Once finished the farmer can be taken to the farmer's dashboard where they can add stock to their live store where users can buy the products that they (farmers) would have put on the live market
  - ▶ They also have an inventory page where they can view their inventories and contact GLH page.
  - ▶ In the nav bar they can view their profile and logout of the farmer portal
  - ▶ The move products page and order fulfilment page are still under development

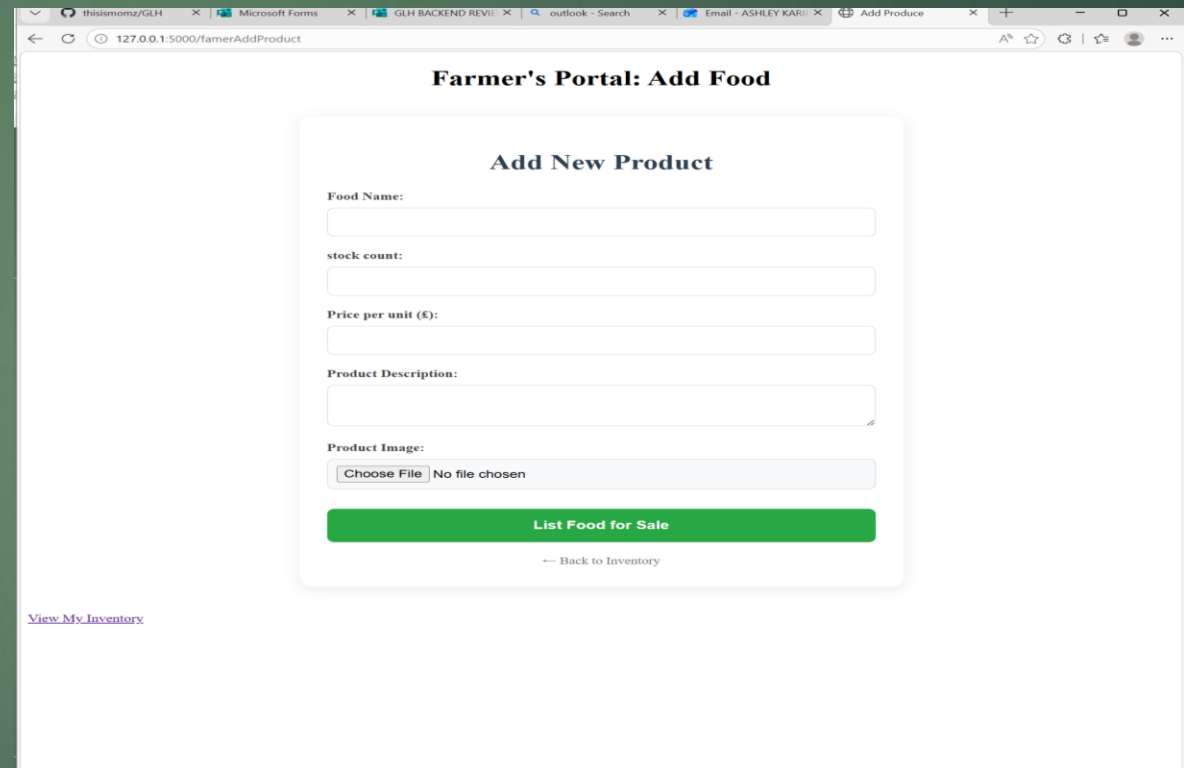


# Farmers nav bar profile dropdown



## Add new products

The farmer can add food to their live store by clicking my profile – add new products and after hey can list the food they want to add

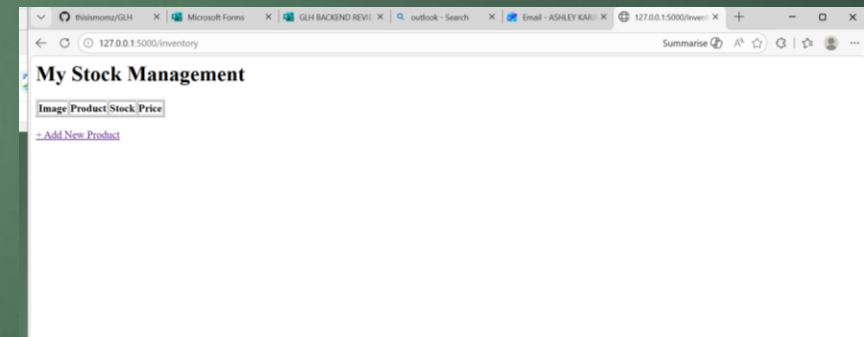
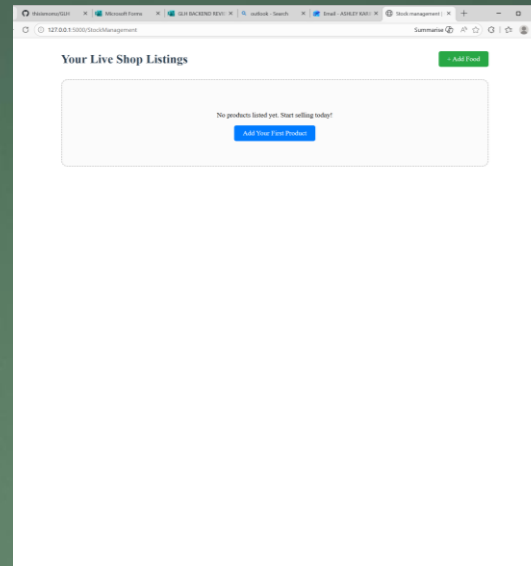


The screenshot shows a web browser window with the title "Farmer's Portal: Add Food". The browser's address bar shows the URL "127.0.0.1:5000/famerAddProduct". The page contains a form titled "Add New Product" with the following fields:

- Food Name:
- stock count:
- Price per unit (£):
- Product Description:
- Product Image:  No file chosen

Below the form is a green button labeled "List Food for Sale" and a link labeled "← Back to Inventory". In the bottom left corner of the page, there is a link labeled "View My Inventory".

The new product the farmer would have added can be found in their inventory where here they can add and not remove but in the stock management page they can remove the products



Logged in users can purchase what ever item that was added to the moment the famer adds it to their live store and when the food is out of stock it will remove form the market and the farmer will know that they are now out of stock

   127.0.0.1:5000/inventory

## My Stock Management

| Image                                                                               | Product | Stock | Price  |
|-------------------------------------------------------------------------------------|---------|-------|--------|
|  | apple   | 0     | \$1.49 |

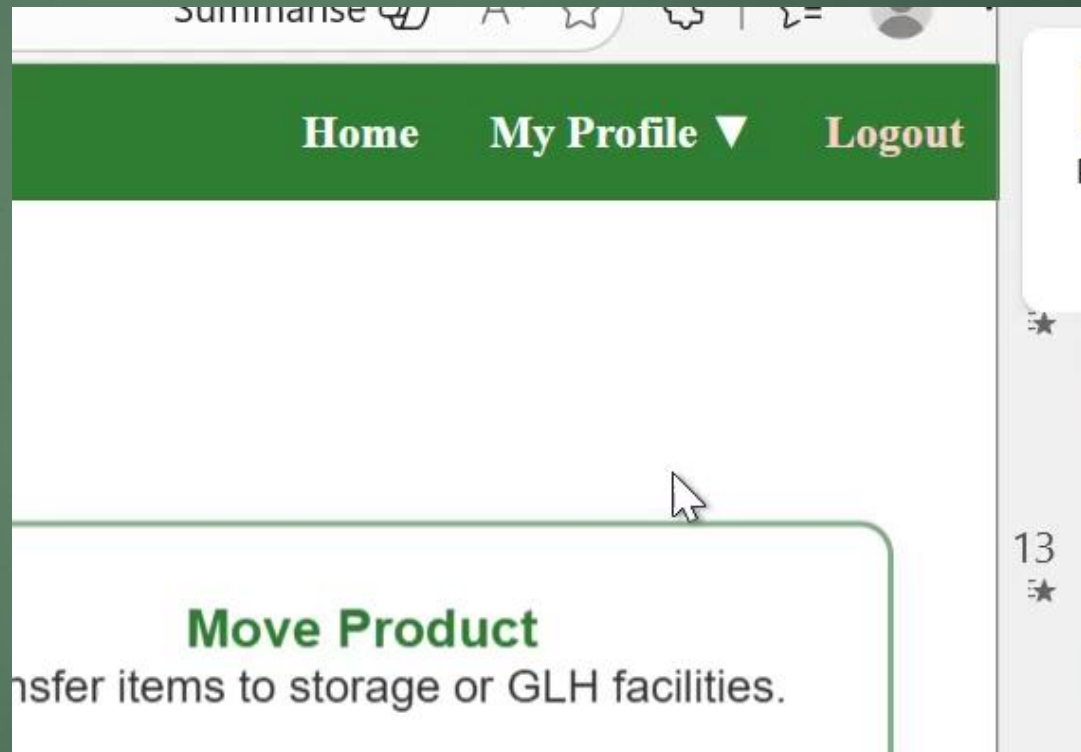
[+ Add New Product](#)



The rest of drop down option

These include certification ,  
Verification and payment  
details

Verification being the  
business one we have gone  
through before



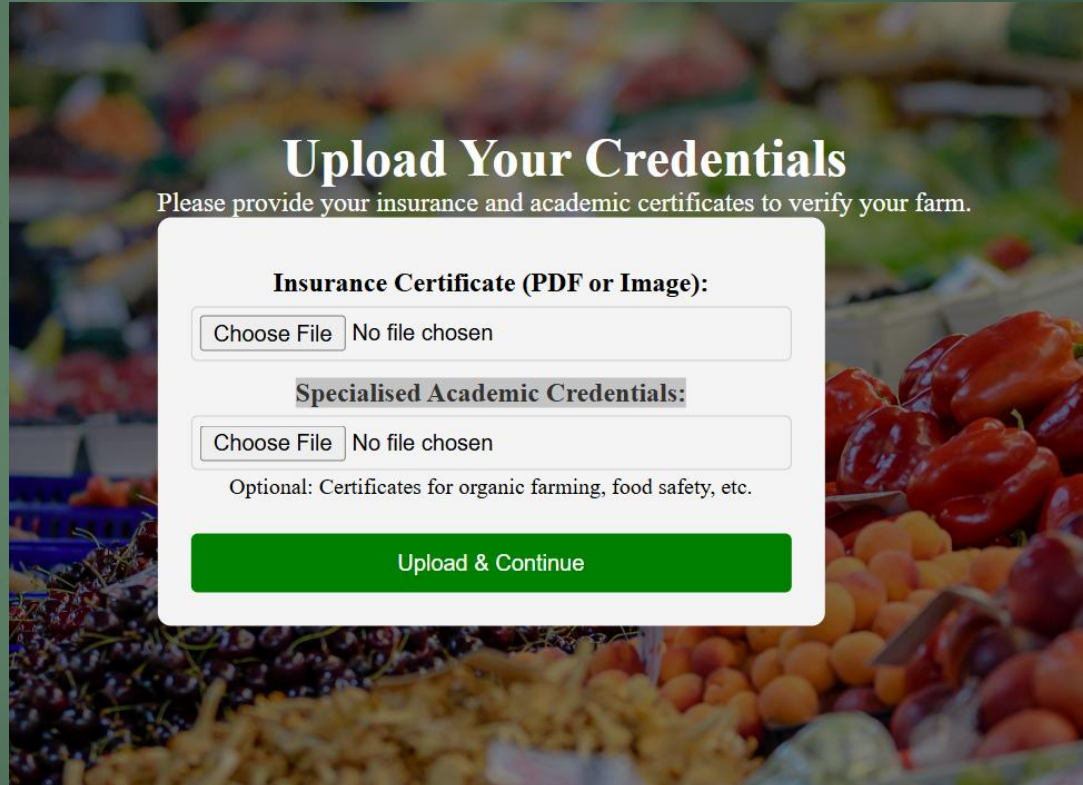
# Certification

Once the farmer clicks this option they will be taken to a page where they have to upload their credential this will include their

**Insurance Certificate**

**And**

**Specialised Academic Credentials**



**Upload Your Credentials**  
Please provide your insurance and academic certificates to verify your farm.

**Insurance Certificate (PDF or Image):**

No file chosen

**Specialised Academic Credentials:**

No file chosen

Optional: Certificates for organic farming, food safety, etc.

# Payment details

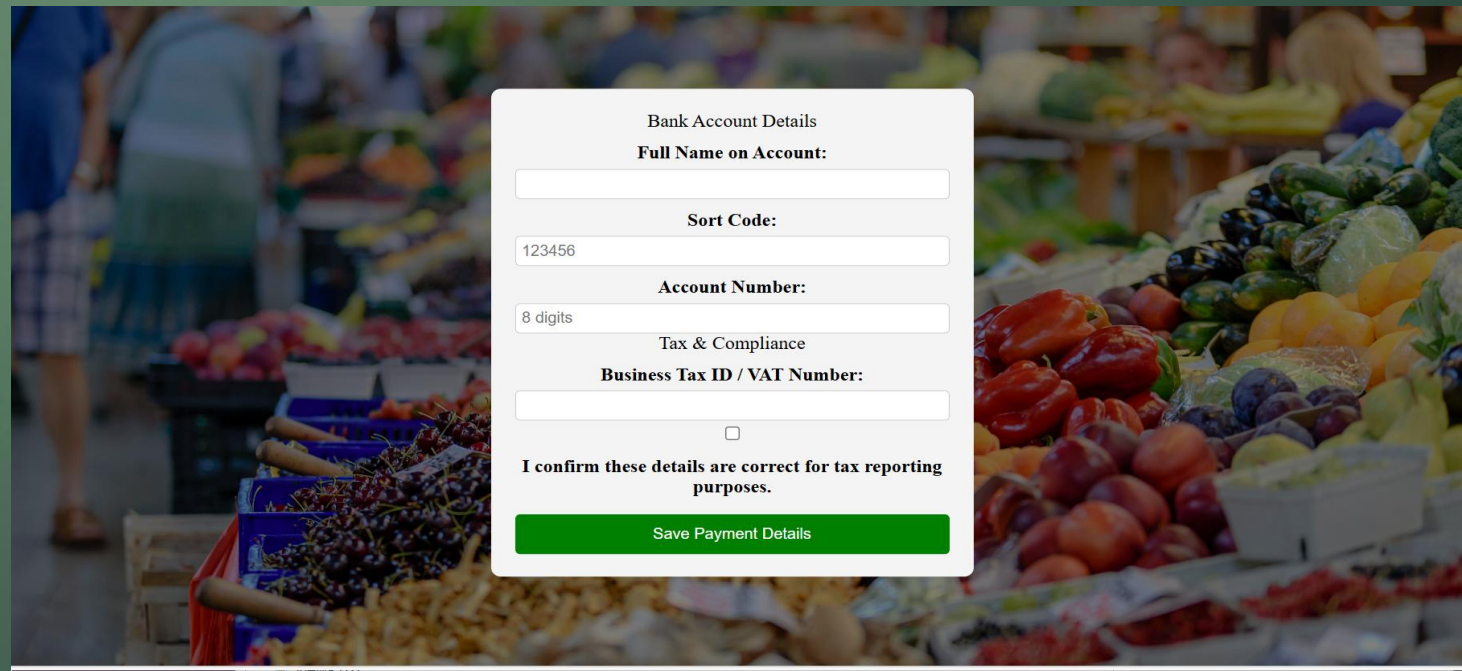
This is a means/way for GLH to pay the farmer and add when orders have been made or sales going through the information stored on this page is stored in a secure file and these details are too important

```
@app.route('/payfarmers', methods=['GET', 'POST'])
def payfarmers():
    if request.method == 'POST':
        # Get data from the form
        acc_name = request.form.get('account_name')
        sort = request.form.get('sort_code')
        acc_num = request.form.get('account_number')
        tax = request.form.get('tax_id')
        username = session.get('user_onboarding') or session.get('username')

        if username:
            # Create a simple string of the farmer's details
            farmer_data = (
                f"Username: {username}\n"
                f"Account Name: {acc_name}\n"
                f"Sort Code: {sort}\n"
                f"Account Number: {acc_num}\n"
                f"Tax ID: {tax}\n"
                f"{'-'*20}\n"
            )

            # Append the details to a local text file
            try:
                with open('farmer_details.txt', 'a') as f:
                    f.write(farmer_data)
                print(f"Details for {username} saved to file.")
                return redirect(url_for('Dashboard'))
            except Exception as e:
                print(f"Error saving to file: {e}")
        else:
            print("No username in session. Please log in again.")

    return render_template('payfarmers.html')
```



Bank Account Details

**Full Name on Account:**

**Sort Code:**

**Account Number:**

**Tax & Compliance**

**Business Tax ID / VAT Number:**

☐

I confirm these details are correct for tax reporting purposes.

Save Payment Details



That concludes the  
farmer dashboard  
as of now

# The about page

- ▶ This page tell the user what GLH means what they stand for and what they are about
- ▶ This page can be accessed by anyone as they do not need any login or permissions to view it is open to all the public

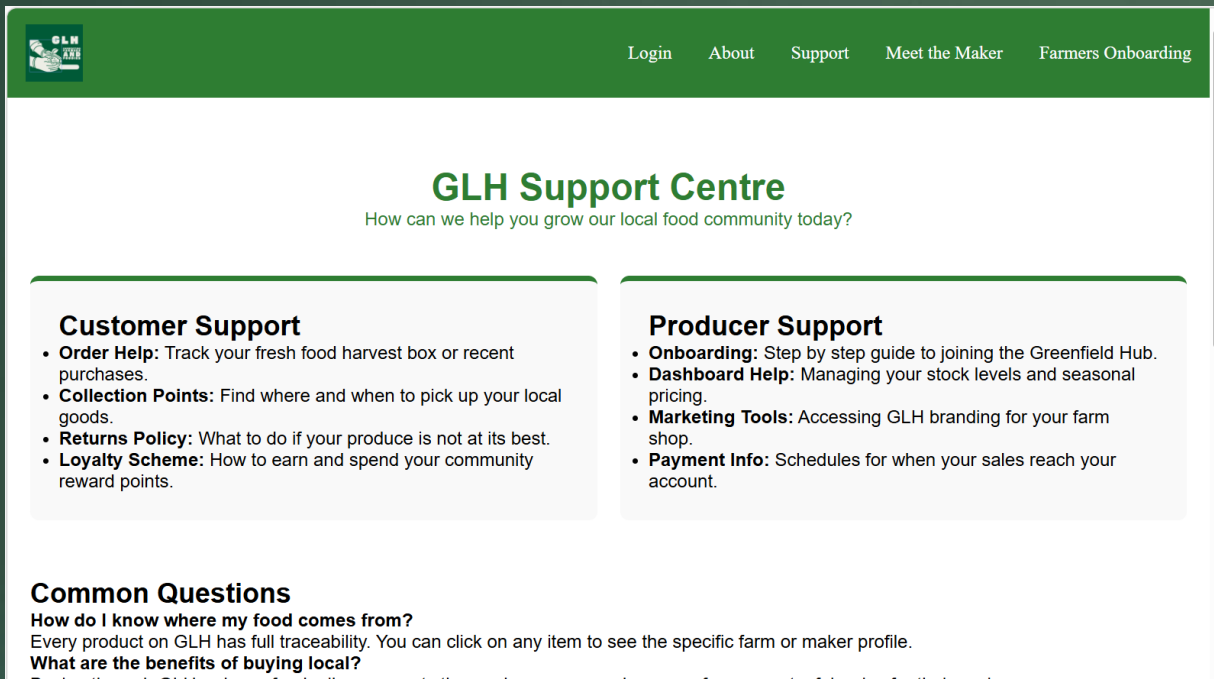


- Users can navigate to the about page via the navigation bar



# Support page

- ▶ This page gives information to any user who has any problem with the web application they do not need to log in just in case they had login problems



- There is a form that the customers can access at the bottom of this page as well

Buying through GLH reduces food miles, supports the rural economy, and ensures farmers get a fair price for their work.

### Send us a Message

I am a Customer ▾

How can we help?

Submit Request

Prefer to speak with us? Call the hub at **01480 890 970**  
Support Hours: Monday to Friday, 9am to 5pm

# Browse the Market

- ▶ This is the page where the uploaded items by the farmers can be found
- ▶ Users can add items to check out view their cart and purchase their items
- ▶ We also intend to add a history cart in later updates

Fresh From The Farms Of GLH

View Cart



NZUNGU  
NICE  
£12.0

Add to Cart



apple  
it taste good  
£123.0

Add to Cart

Your Shopping Cart



apple  
£123.00

- 2 +

£246.00

Grand Total: £246.00  
[Add More](#) [Purchase Now](#)

Finalise Your Order

Full Name

John Doe

Delivery Address

123 Farm Road...

Email

john@example.com

PLACE ORDER

[Return to Cart](#)



# Admin portal

- ▶ This page cannot be accessed by any user from the normal home page unless they already have an admin account
- ▶ To register as an admin they must go to the [Secure Admin Login](#)
- ▶ Where they will be required to enter the code that will register their account (YOUR\_MASTER\_CODE\_123)

## Admin Portal

**Admin Username**

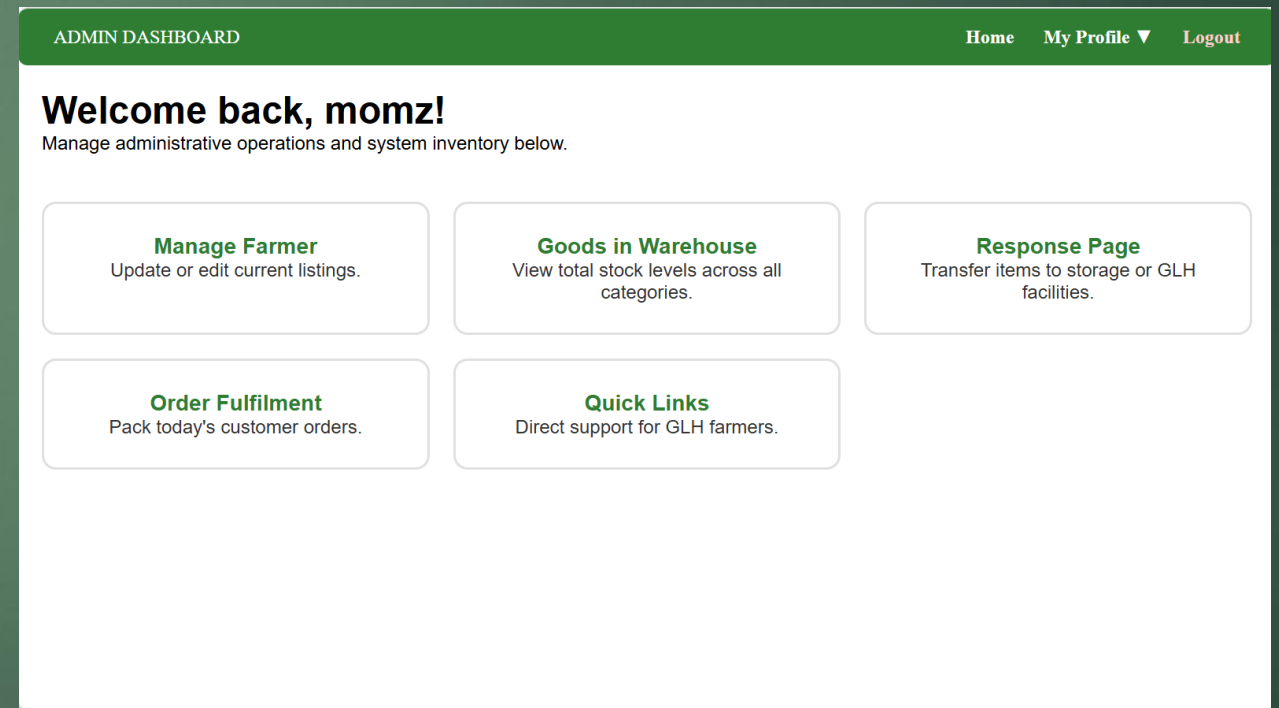
**Password**

**Master Security Code**

Enter the secret code

**Unlock Dashboard**

- ▶ And after they have signed in, they will be taken to the admin dashboard
- ▶ This part of the site was made with the GLH admin team in mind



# This page is currently InProgress

- ▶ On this dashboard they can
  - ▶ Manage Farmers
  - ▶ See all the goods that where sent to their website
  - ▶ Respond to all the farmers/ user problems
  - ▶ See all the orders that have taken place
  - ▶ They also have a link to all the site if any issues occure

► Thank you